

G02 / PROTECTED ANSWERS

G02 REVIEW A

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

YOUR STANDALONE GATE BOOK 3

Open only after Gate A attempt is saved 3

Source contract and independent totals 3

Order-grain preparation 3

Region and average reasoning 4

Join and distinct-fee failures 4

Highest versus latest 5

Set logic and absence 5

Database thinking: evidence, not a copied error sentence 6

Changed micro-tasks: repair only the dimensions that need it 6

Repair review: open after saving the micro-task 7

Fresh full reassessment 7

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

YOUR STANDALONE GATE BOOK

Open only after Gate A attempt is saved

Assessment rule: Save the current attempt before opening its review. Fresh Gate B stays closed until explicitly assigned after repair.

Source contract and independent totals

The source has four customers, five orders and six item lines. Item gross totals 350 and order-header fees total 32, so all-order gross must reconcile to 382. There are two known-paid orders, two known-unpaid and one unknown-payment order. December 1 includes G501 and G502. A half-open day filter uses $\geq \text{DATE '2026-12-01'}$ and $< \text{DATE '2026-12-02'}$.

```
SELECT COUNT(*) AS orders, SUM(shipping_fee) AS fees
FROM course.g02_orders;
```

```
SELECT COUNT(*) AS lines, SUM(quantity * unit_price) AS item_gross
FROM course.g02_items;
```

These checks come directly from original sources. Re-summing a wrong joined result twice would not be an independent reconciliation.

Order-grain preparation

```
CREATE TEMP VIEW g02_order_result AS
WITH items AS (
  SELECT order_id, SUM(quantity * unit_price) AS item_gross
  FROM course.g02_items GROUP BY order_id
)
SELECT o.order_id, o.customer_id, o.ordered_on,
       COALESCE(c.region, 'Unknown') AS region_group,
       CASE WHEN o.is_paid = TRUE THEN 'Paid'
            WHEN o.is_paid = FALSE THEN 'Unpaid'
            ELSE 'Unknown' END AS payment_group,
       COALESCE(i.item_gross, 0) AS item_gross,
       o.shipping_fee,
       COALESCE(i.item_gross, 0) + o.shipping_fee AS order_gross
FROM course.g02_orders AS o
LEFT JOIN items AS i ON o.order_id = i.order_id
LEFT JOIN course.g02_customers AS c
  ON o.customer_id = c.customer_id;

SELECT *
FROM g02_order_result
ORDER BY order_id;

SELECT COUNT(*) AS ROWS, COUNT(DISTINCT order_id) AS ids,
       SUM(item_gross) AS items, SUM(shipping_fee) AS fees, SUM(order_gross) AS total
FROM g02_order_result;
```

The temporary view is only a convenient way for this review to reuse the CTE result. A correct learner solution may repeat CTEs instead. Do not require this exact spelling when the meaning and checks are correct.

Order	Payment	Items	Fee	Gross
G501	Paid	90.00	12.00	102.00
G502	Unpaid	80.00	12.00	92.00
G503	Unknown	50.00	0.00	50.00
G504	Paid	130.00	8.00	138.00
G505	Unpaid	0.00	0.00	0.00

The validation query returns five rows, five distinct IDs, items 350, fees 32 and total 382. G505 remains despite having no item rows and no customer identity. Unknown payment on G503 stays separate from G502 and G505's known FALSE states.

Region and average reasoning

```
SELECT region_group, COUNT(*) AS orders, SUM(item_gross) AS items,
       SUM(shipping_fee) AS fees, SUM(order_gross) AS gross,
       ROUND(AVG(order_gross), 2) AS avg_order_gross
FROM g02_order_result
GROUP BY region_group
ORDER BY region_group;
```

```
SELECT region_group, SUM(order_gross) AS gross
FROM g02_order_result
GROUP BY region_group
HAVING SUM(order_gross) >= 100
ORDER BY region_group;
```

East: two orders, items 140, fees 12, gross 152, mean 76. North: one order, 130/8/138, mean 138. Unknown: one empty order, 0/0/0, mean 0. West: one order, 80/12/92, mean 92.

Thresholded groups are East 152 and North 138, total 290. Their 92 difference from the full 382 is the intentionally excluded West group; Unknown contributes zero amount but one excluded order.

The all-order mean is $382 / 5 = 76.40$. A simple average of the four region means would be 76.50, a deceptively small but real error caused by unequal counts. Passing requires the correct denominator, not merely a number close to the answer.

Join and distinct-fee failures

```
SELECT COUNT(*) AS joined_rows, COUNT(DISTINCT o.order_id) AS represented_orders,
       SUM(o.shipping_fee) AS repeated_fees
FROM course.g02_orders AS o
JOIN course.g02_items AS i ON o.order_id = i.order_id;
```

```
SELECT SUM(DISTINCT shipping_fee) AS wrong_distinct_fee_sum
FROM course.g02_orders;
```

The raw inner join gives six rows, four represented orders and fees 52. G501's fee 12 and G504's fee 8 each occur twice, inflating the sum by 20. G505 is dropped. SUM(DISTINCT shipping_fee)

gives 20, not 32, because G501 and G502 both genuinely have fee 12. Identity comes from order_id, not the numeric charge.

Highest versus latest

```
WITH ranked AS
  (SELECT *,
    ROW_NUMBER() OVER (PARTITION BY customer_id
      ORDER BY order_gross DESC,
      order_id) AS rn
    FROM g02_order_result
    WHERE customer_id IS NOT NULL)
SELECT customer_id, order_id, order_gross
FROM ranked
WHERE rn = 1
ORDER BY customer_id;
```

```
WITH ranked AS
  (SELECT *,
    ROW_NUMBER() OVER (PARTITION BY customer_id
      ORDER BY ordered_on DESC,
      order_id DESC) AS rn
    FROM g02_order_result
    WHERE customer_id IS NOT NULL)
SELECT customer_id, order_id, ordered_on
FROM ranked
WHERE rn = 1
ORDER BY customer_id;
```

Highest results are K01 G501/102, K02 G502/92 and K03 G504/138. Latest results are K01 G503 on December 2, K02 G502 on December 1 and K03 G504 on December 3. K01 demonstrates why value ordering and date ordering are different requirements. The specified tie rules should still be present even when this fixture does not force a latest-date tie.

Set logic and absence

```
SELECT customer_id
FROM course.g02_orders
WHERE is_paid = TRUE
UNION ALL
SELECT customer_id
FROM course.g02_orders
WHERE is_paid IS NULL
ORDER BY customer_id;
```

```
SELECT customer_id
FROM course.g02_orders
WHERE is_paid = TRUE
UNION
SELECT customer_id
FROM course.g02_orders
WHERE is_paid IS NULL
ORDER BY customer_id;
```

```
SELECT c.customer_id
FROM course.g02_customers AS c
WHERE NOT EXISTS
  (SELECT 1
    FROM course.g02_orders AS o
    WHERE o.customer_id = c.customer_id)
ORDER BY c.customer_id;
```

UNION ALL returns K01 twice and K03 once. UNION returns K01 and K03 once each. The repeated K01 arose from a paid order and an unknown-payment order; the chosen selected field was customer_id. K04 is the customer with no orders. No nullable NOT IN list is needed to express that absence.

Database thinking: evidence, not a copied error sentence

A valid disposable contract can be quantity integer NOT NULL CHECK (quantity > 0), with a separate primary key. A duplicate key should produce SQLSTATE 23505, a negative quantity 23514, and a null required quantity 23502. The precise human-readable message includes your actual table and constraint names. Keep the error you actually observed.

An explicit BEGIN, valid UPDATE, SELECT, ROLLBACK, SELECT should show the changed value before rollback and the original after it. If a statement fails inside the transaction, first recover with ROLLBACK. Your connection query should name your actual standalone_de database and current role. A plan explanation should distinguish EXPLAIN estimates from EXPLAIN ANALYZE's executed observations; no fixed scan type or timing is required for a tiny source.

Do not record level 3 here merely because these messages are listed in the book. You need your own rejected writes, preserved original record, transaction result and explanation. This is one of the seven independently scored dimensions.

Changed micro-tasks: repair only the dimensions that need it

Use course.g02_repair_customers, course.g02_repair_orders and course.g02_repair_items. These are changed data, not full gate variants B or C. Start each relevant repair from a blank editor and save it separately.

Dimension needing repair	Fresh independent micro-task
Foundation SQL semantics	List known-paid, known-unpaid and unknown orders separately; select January 2 with a half-open day filter.
Transformation + grouping	Build payment-group gross totals, then keep groups with gross at least 40.
Joins + set logic	Compare inner-join row coverage with all orders; compare UNION and UNION ALL on a repeated source selection; find customers without orders.
CTEs + windows	Prepare order totals, then compute a running gross in date/ID order with an explicit ROWS frame.
Multi-grain reconciliation	Retain every order and prove item total plus original fees equals final gross. Explain why a distinct fee sum fails.
PostgreSQL thinking	Create a new temporary contract, deliberately reject a bad row, demonstrate rollback, and explain one read-only plan.
Evidence + explanation	Save the first repair before review; record actual results, help and a grain/null/denominator explanation in your own words.

Repair review: open after saving the micro-task

H601 has item gross 30, fee 7 and order gross 37; H602 has 40, 7 and 47; H603 is empty with fee zero and order gross zero. Three order rows total 84, made of items 70 plus fees 14. A distinct fee sum would give 7. Paid gross is 37, Unknown 47 and Unpaid 0; only Unknown passes the ≥ 40 group threshold. January 2 contains H602. J03 has no orders.

The raw inner join has three item rows representing two orders. It repeats fee 7 for H601 and shows joined fee sum 21. A left join to raw items gives four rows representing all three orders. A running all-order gross by date and ID is 37, 84, 84. The all-order mean is 28, while the two non-empty orders average 42. Both can be valid under their different declared populations.

Reassess each weak dimension using the same level-3 standard and actual fresh evidence. If the solution has become a memorised sequence and the changed task no longer tests independence, record that honestly and obtain a deliberately new full assessment when ready. Do not silently relabel these micro-tasks as protected variants B or C.

Fresh full reassessment

If a full reassessment is assigned after repair, use the sealed G02-B brief and its separate setup script. The repair micro-tasks above are not Gate B.